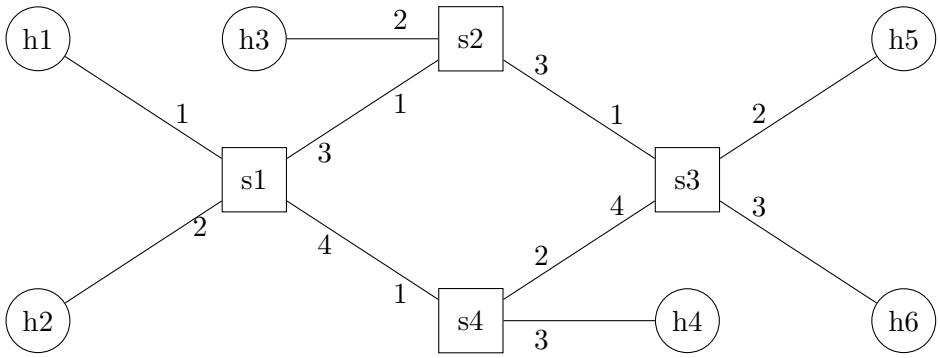
Software Defined Networking – 29 settembre 2019

Esercizio 1

Si consideri la rete in figura. I nodi h1,..., hn contengono tabelle ARP prepopolate per tutta la rete. Gli indirizzi MAC e IP dei nodi h1,..., hn sono MAC1,..., MACn e IP1,..., IPn. Sui nodi h1,..., hn sono in esecuzione processi in ascolto su tutte le porte TCP e UDP di interesse. Il controllore Ryu è collegato ai nodi s1,..., sm ed è configurato come indicato nel Programma 1. Le interfacce tra gli switch e il controllore non sono indicate in figura. Se non altrimenti specificato, il TTL iniziale è 200.



Nelle risposte usare per ogni pacchetto il seguente formato:

Nodo → interfaccia: MAC sorgente → MAC destinazione; ethertype; IP sorgente → IP destinazione; TTL; protocollo; porta sorgente → porta destinazione;

Negli switch sono preinstallate le regole in Figura 1.

L'host h1 porta 1234 apre una connessione TCP verso h5 porta 5678.

(a) Scrivere tutti i pacchetti uscenti da tutte le interfacce di s2.

(b) Dire quanti pacchetti arrivano in h5 e che valore ha il campo TTL.

L'host h2 porta 3322 apre una connessione TCP verso h3 porta 123.

(c) Scrivere tutti i pacchetti uscenti da tutte le interfacce di s1.

(d) Dire quanti pacchetti arrivano in h3 e con che TTL.

(e) L'host h3 risponde. Scrivere tutti i pacchetti uscenti da tutte le interfacce di s1. Scrivere il contenuto della tabella openflow in s1.

(f) Scrivere quanti pacchetti arrivano in h2 e che valore ha il campo TTL.

L'host h5 porta 7777 invia un datagramma UDP verso l'host h6 porta 245.

(g) Scrivere le regole che il controllore deve inserire in s3 per consentire questo scambio di pacchetti. Nessuno di questi pacchetti deve arrivare agli altri host.

Figura 1: Regole iniziali

Switch s1		
Priority	Match	Action
0	*	DecTTL Output(FLOOD)
5	eth_src = MAC2	Output(CONTROLLER,128)
5	eth_dst = MAC2	Output(CONTROLLER,128)

Switch s2		
Priority	Match	Action
0	*	Output(CONTROLLER,128)
5	eth_type = IPv4 ip_proto = TCP	DecTTL Output(FLOOD)

Switch s3		
Priority	Match	Action
0	*	DecTTL Output(FLOOD)
5	eth_src = MAC6	Output(CONTROLLER,128)
5	eth_dst = MAC6	Output(CONTROLLER,128)

Switch s4		
Priority	Match	Action
0	*	Output(CONTROLLER,128)
5	eth_type = IPv4 ip_proto = UDP	Output(FLOOD)

Programma 1 – Packet In

```
@set_ev_cls(ofp_event.EventOFPPacketIn, MAIN_DISPATCHER)
def packet_in_handler(self, ev):
    msg = ev.msg; datapath = msg.datapath; ofproto = datapath.ofproto;
    parser = datapath.ofproto_parser; in_port = msg.match['in_port']
    pkt = packet.Packet(msg.data); pkt_eth = pkt.get_protocol(eth.eth)
    pkt_ipv4 = pkt.get_protocol(ipv4.ipv4)
    pkt_udp = pkt.get_protocol(udp.udp); pkt_tcp = pkt.get_protocol(tcp.tcp)
    buffer_id = msg.buffer_id; data = msg.data

    if (datapath.id == 1 and pkt_tcp is not None):
        match = parser.OFPMatch(eth_dst=pkt_eth.dst, eth_type=ETH_TYPE_IP,
            ipv4_dst = pkt_ipv4.dst, ip_proto=IPPROTO_TCP,
            tcp_dst=pkt_tcp.dst_port)
        actions = []
        if(pkt_tcp.dst_port<1024):
                  actions.append( parser.OFPDecTTL() )
        actions.append( parser.OFPActionOutput(FLOOD) )
        inst = [parser.OFPInstructionActions(ofproto.OFPIT_APPLY_ACTIONS,actions)]
        mod = parser.OFPFlowMod(datapath=datapath, priority=10,
            match=match, instructions=inst, buffer_id=buffer_id)
        datapath.send_msg(mod)
```

Esercizio 2

Un filtro di Bloom *a conteggio* è costituito da un array $A[_]$ di m numeri interi e k funzioni hash. Nel filtro *a conteggio* ogni volta che viene inserito un elemento x, le celle di A in posizione $h_i(x)$ con $1 \leq i \leq k$ sono incrementate di 1. Nel filtro *a conteggio* è possibile rimuovere un elemento decrementando le celle corrispondenti.

Un sistema di filtraggio delle connessioni TCP usa un filtro di Bloom a conteggio. Nel momento in cui una connessione TCP è autorizzata i suoi parametri sono inseriti nel filtro. Al termine della connessione gli stessi parametri sono rimossi. Si supponga che il filtro contenga al massimo 10 000 connessioni.

In filtro è costituito da un array di $m = 2^{10}$ celle e $k = 3$ funzioni hash.

(a) In un dato momento, nel filtro sono presenti 100 connessioni. Qual è la probabilità che una nuova connessione non autorizzata sia ammessa nel sistema.

(b) Calcolare la grandezza minima del filtro (in bit) per non avere mai overflow.

(c) Calcolare il numero minimo di celle dell'array per avere un probabilità di falso positivo sempre inferiore a 10^{-4}.

Si considerino $m = 19$, $k = 2$ e la famiglia di funzioni hash $h_i(x) = 2i + 3x \bmod m \quad$ per $1 \leq i \leq k$

(d) Mostrare lo stato del filtro dopo l'inserimento degli elementi 3 e 17.

(e) Mostrare lo stato del filtro dopo la rimozione dell'elemento 10. Verificare se gli elementi 3 e 7 sono presenti.

Esercizio 3

Una matrice di commutazione crossbar di uno switch 3×3 usa un algoritmo per lo scheduling delle trasmissioni. Si assumano pacchetti tutti di lunghezza L. Al tempo $t = 0$ sono presenti i seguenti pacchetti, in ordine dal più recente al più vecchio. Ogni pacchetto è etichettato con il numero dell'interfaccia di uscita cui è diretto. L'algoritmo di assegnazione delle risorse obbedisce alla seguente regola: *si sceglie sempre l'interfaccia con l'identificativo minore.*

Ingresso 1: 1, 3, 2

Ingresso 2: 2, 1, 3

Ingresso 3: –, 3, 2

(a) Svolgere l'algoritmo Take-a-Ticket (TaT) fino allo svuotamento di tutte le code, specificando ad ogni round quali pacchetti sono trasmessi.

(b) Svolgere l'algoritmo Parallel-Iterative-Matching (PIM) fino allo svuotamento di tutte le code, specificando ad ogni round quali pacchetti sono trasmessi.

Si consideri il caso in cui alcuni pacchetti abbiano lunghezza $2L$.

(c) Dire se è possibile usare l'algoritmo TaT. In caso negativo dire quali modifiche sono necessarie al sistema per usare pacchetti di lunghezza variabile.

(d) Dire se è possibile usare l'algoritmo PIM. In caso negativo dire quali modifiche sono necessarie al sistema per usare pacchetti di lunghezza variabile.

Esercizio 4

Si consideri un sistema di misura del traffico. Siano date le funzion hash h_1 e h_2, con output di $m = 8$ bit. Ad ogni pacchetto è associato un *flow id* F tale che connessioni TCP distinte abbiano flow id distinti. Sia L la lunghezza del pacchetto in ottetti.

In ogni nodo sono presenti due array A di 2^m elementi che sono azzerati ogni $T = 15$ minuti. Ad ogni pacchetto, il nodo incrementa incrementa di L gli elementi $A_i[h_i(F)]$, con $i = 1, 2$. Se, dopo l'incremento, entrambi i valori $A_i[h_i(F)]$ sono maggiori di una soglia S, il *flow id* F è salvato su disco.

I pacchetti arrivano a velocità $C = 1\,\text{Gbit/s}$.

(a) Calcolare la grandezza minima (in bit) degli array A in modo da garantire che non ci sia overflow.

(b) Calcolare la soglia S in modo da salvare i flussi che, nel tempo T, hanno mostrato un throughput di almeno $1\,\text{Mbit/s}$.

(c) Ad un certo punto, sono arrivate al sistema $n = 100$ connessioni da $1\,\text{Mbit/s}$. Un pacchetto appartenente a una nuova connessione si presenta al sistema. Stimare la probabilità che il *flow id* di tale pacchetto sia salvato su disco.